

CarRent + role i rejestracja

Wiele ról użytkownika, Spring Security, JWT i rejestracja

**ROLE_ADMIN /
ROLE_USER**

Many-to-Many

users

id
login
password_hash

user_roles

user_id
role_id

roles

id
name

Zakres zmian

Co dokładamy do istniejącego Spring Security + JWT

Przed zmianą

users.role jako jedna wartość
Role jako enum: USER, ADMIN
JWT claim: role
Authority budowane z jednej roli

Zmiana

Role jako encja JPA
users ↔ roles przez user_roles
User ma Set<Role>
JWT claim: roles
Rejestracja nadaje ROLE_USER

W wykładzie używamy tylko JPA. Nie musimy przechowywać implementacji JDBC, JSON ani Hibernate w aplikacji zaliczeniowej.

Model danych: wiele ról

Jeden użytkownik może mieć kilka ról, a jedna rola może należeć do wielu użytkowników

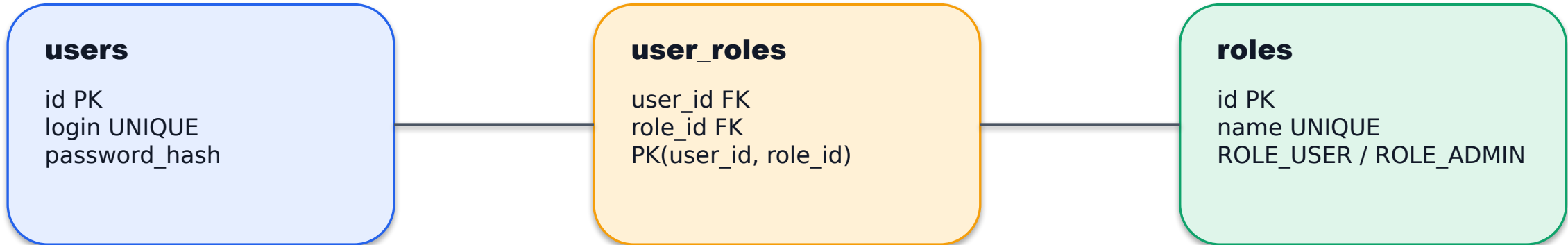


Tabela user_roles nie jest osobną encją w aplikacji. To tabela łącząca relację @ManyToMany.

SQL: tabele ról

Manualny schemat bazy zgodny z mapowaniem JPA

```
CREATE TABLE users (  
    id TEXT PRIMARY KEY,  
    login TEXT NOT NULL UNIQUE,  
    password_hash TEXT NOT NULL  
);  
  
CREATE TABLE roles (  
    id TEXT PRIMARY KEY,  
    name TEXT NOT NULL UNIQUE  
);  
  
CREATE TABLE user_roles (  
    user_id TEXT NOT NULL,  
    role_id TEXT NOT NULL,  
    PRIMARY KEY (user_id, role_id),  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE  
);
```

SQL: dane startowe

Role mają nazwy authority Spring Security

```
INSERT INTO roles (id, name) VALUES
('1', 'ROLE_ADMIN'),
('2', 'ROLE_USER');

INSERT INTO users (id, login, password_hash) VALUES
('fc6e600b-e89e-42e2-b2b7-ca18edb68924', 'lukasz', '...hash...'),
('f4b5b79b-df2c-4e0a-89fc-464908d2e6dc', 'kamil', '...hash...');

INSERT INTO user_roles (user_id, role_id) VALUES
('fc6e600b-e89e-42e2-b2b7-ca18edb68924', '1'),
('fc6e600b-e89e-42e2-b2b7-ca18edb68924', '2'),
('f4b5b79b-df2c-4e0a-89fc-464908d2e6dc', '2');
```

hasRole("ADMIN") sprawdza authority ROLE_ADMIN, dlatego w tabeli roles.name trzymamy pełną nazwę ROLE_*.

SQL: pojazdy i wypożyczenia

Zmiana ról nie usuwa danych domenowych aplikacji

vehicle

id
category, brand, model
production_year, plate, price
attributes JSONB

rental

id
vehicle_id FK
user_id FK
rent_date, return_date

```
INSERT INTO vehicle (...) VALUES  
(  
  '1', 'Bus', 'Volkswagen', 'T2', 1985, 'LU123', 100.0, '{"seats": 20}'  
,  
  '2', 'Motorcycle', 'Honda', 'CBR600', 2016, 'LU456', 200.0, '{"licence_category": "A"}'  
,  
  '3', 'Car', 'Toyota', 'Corolla', 2024, 'LU789', 300.0, '{"fuelType": "hybrid"}'  
);
```

Wypożyczenia nadal wskazują users.id. Relacja ról nie zmienia Rental.

application-jpa.yml

Jeżeli baza jest tworzona skryptem, JPA powinno walidować schemat

```
spring:
  datasource:
    url: ${DB_URL}
    driver-class-name: org.postgresql.Driver

  jpa:
    hibernate:
      ddl-auto: validate
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
```

Dlaczego validate?

Baza jest jawnie opisana w SQL
Aplikacja sprawdza zgodność encji z tabelami
Mniejsze ryzyko cichego dopisania złego schematu

W dev można użyć update

Szybkie prototypowanie
Mniej SQL na starcie
Ale łatwiej ukryć błąd mapowania

Encja Role

Role przestają być enumem i stają się rekordami w tabeli roles

```
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
@EqualsAndHashCode(of = "id")
@ToString(exclude = "users")
@Entity
@Table(name = "roles")
public class Role {

    @Id
    @Column(nullable = false, unique = true)
    private String id;

    @Column(nullable = false, unique = true)
    private String name; // ROLE_USER, ROLE_ADMIN

    @ManyToMany(mappedBy = "roles")
    @JsonIgnore
    private Set<User> users;
}
```

Ważne

Role.name to pełna authority
Nie trzymamy USER/ADMIN
Trzymamy ROLE_USER/ROLE_ADMIN
equals/hashCode bez users

User: relacja @ManyToMany

Użytkownik posiada zbiór ról zamiast pojedynczej roli

```
@Entity
@Table(name = "users")
public class User {

    @Id
    private String id;

    @Column(nullable = false, unique = true)
    private String login;

    @JsonIgnore
    @Column(name = "password_hash", nullable = false)
    private String passwordHash;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles;
}
```

Mapowanie

users.id → user_roles.user_id
roles.id → user_roles.role_id
JPA zapisuje pary w user_roles
Security czyta rolę z user.getRoles()

User.copy()

Po zmianie modelu trzeba kopiować Set<Role>, nie stare pole role

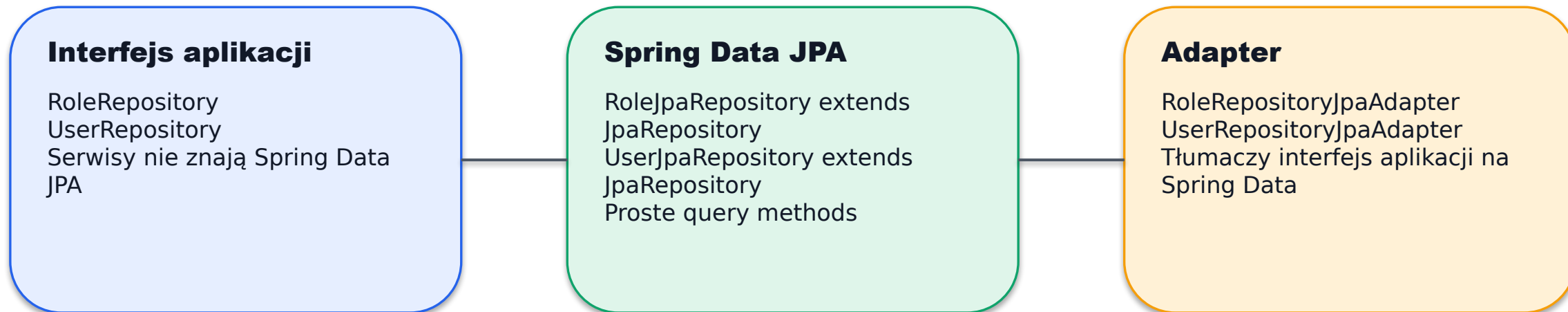
```
public User copy() {  
    return User.builder()  
        .id(id)  
        .login(login)  
        .passwordHash(passwordHash)  
        .roles(roles == null  
            ? new HashSet<>()  
            : new HashSet<>(roles))  
        .build();  
}
```

Nie .roles(roles)

Kopia dostałaby tę samą referencję do seta
Zmiana w kopii mogłaby zmienić oryginał
HashSet zabezpiecza przed przypadkowym
współdzieleniem

Repozytoria: zostaje wzorzec z adapterem

Serwisy nadal mogą zależeć od własnych interfejsów repozytoriów



Jeżeli aplikacja będzie już wyłącznie JPA, można uprościć i używać Spring Data bez adaptera.

RoleRepository

Wspólny kontrakt potrzebny głównie przy rejestracji

```
public interface RoleRepository {  
    List<Role> findAll();  
  
    Optional<Role> findById(String id);  
  
    Optional<Role> findByName(String name);  
  
    Role save(Role role);  
}
```

findByName

Rejestracja szuka ROLE_USER
Nie zapisujemy roli w kodzie
Rola musi istnieć w tabeli roles

```
roleRepository.findByName("ROLE_USER")
```

RoleJpaRepository

Spring Data JPA robi większość pracy za nas

```
@Profile("jpa")
public interface RoleJpaRepository
    extends JpaRepository<Role, String> {

    Optional<Role> findByName(String name);
}
```

JpaRepository daje od razu

```
findAll()
findById(id)
save(entity)
deleteById(id)
```

findByName jest generowane przez Spring Data na podstawie nazwy metody i pola name w encji Role.

RoleRepositoryJpaAdapter

Adapter implementuje nasz interfejs i deleguje do Spring Data

```
@Repository
@Profile("jpa")
public class RoleRepositoryJpaAdapter implements RoleRepository {

    private final RoleJpaRepository delegate;

    public RoleRepositoryJpaAdapter(RoleJpaRepository delegate) {
        this.delegate = delegate;
    }

    public List<Role> findAll() { return delegate.findAll(); }
    public Optional<Role> findById(String id) { return delegate.findById(id); }
    public Optional<Role> findByName(String name) { return delegate.findByName(name); }

    public Role save(Role role) {
        if (role.getId() == null || role.getId().isBlank()) {
            role.setId(UUID.randomUUID().toString());
        }
        return delegate.save(role);
    }
}
```

Efekt

Serwisy zależą od RoleRepository
Implementacja wybierana przez profil
jpa

UserJpaRepository

Przy logowaniu dobrze pobrać użytkownika razem z rolami

```
@Profile("jpa")
public interface UserJpaRepository
    extends JpaRepository<User, String> {

    @EntityGraph(attributePaths = "roles")
    Optional<User> findByLogin(String login);
}
```

@EntityGraph

Wymusza dociągnięcie roles razem z User
Zmniejsza ryzyko
LazyInitializationException
Jest czytelne przy logowaniu

Jeżeli w User.roles FetchType.EAGER, EntityGraph nie jest konieczny, ale dobrze pokazuje intencję: login potrzebuje ról.

UserRepositoryJpaAdapter

Nie trzeba ręcznie zapisywać user_roles

```
@Repository
@Profile("jpa")
public class UserRepositoryJpaAdapter implements UserRepository {

    private final UserJpaRepository delegate;

    public Optional<User> findByLogin(String login) {
        return delegate.findByLogin(login);
    }

    public User save(User user) {
        if (user.getId() == null || user.getId().isBlank()) {
            user.setId(UUID.randomUUID().toString());
        }
        return delegate.save(user);
    }
}
```

@ManyToMany

User.roles jest relacją @ManyToMany
Role istnieją już w tabeli roles
Po save(user) JPA zapisuje wpisy w
user_roles

Uwaga

Nie tworzymy nowej roli ROLE_USER przy
każdym register
Pobieramy istniejącą rolę przez
RoleRepository

MyUserDetailsService

Wszystkie role użytkownika zamieniamy na authorities

```
@Override
public UserDetails loadUserByUsername(String username)
    throws UsernameNotFoundException {

    User user = userRepository.findByLogin(username)
        .orElseThrow(() -> new UsernameNotFoundException(username));

    List<SimpleGrantedAuthority> authorities = user.getRoles().stream()
        .map(role -> new SimpleGrantedAuthority(role.getName()))
        .toList();

    return new org.springframework.security.core.userdetails.User(
        user.getLogin(),
        user.getPasswordHash(),
        authorities
    );
}
```

Nie dodajemy ROLE_ drugi raz

role.getName() zwraca ROLE_ADMIN
hasRole("ADMIN") szuka ROLE_ADMIN
ROLE_ROLE_ADMIN byłoby błędem

Typy generyczne Java

List<SimpleGrantedAuthority>
Konstruktor UserDetails przyjmie tę listę
List<GrantedAuthority> wymagałoby mapowania z typem

JWT: claim roles

Token powinien przechowywać listę ról, nie pojedynczą rolę

```
public String generateToken(UserDetails userDetails) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("roles", getUserRoles(userDetails));

    Date now = new Date();
    Date expirationDate = new Date(now.getTime() + expirationMs);

    return Jwts.builder()
        .claims().add(claims).and()
        .subject(userDetails.getUsername())
        .issuedAt(now)
        .expiration(expirationDate)
        .signWith(getSigningKey())
        .compact();
}

private List<String> getUserRoles(UserDetails userDetails) {
    return userDetails.getAuthorities().stream()
        .map(GrantedAuthority::getAuthority)
        .toList();
}
```

```
{
  "sub": "lukasz",
  "roles": ["ROLE_ADMIN",
"ROLE_USER"],
  "exp": 1779668822
}
```

JwtAuthFilter nadal może łądować aktualne role z bazy przez UserDetailsService. Token nie musi samodzielnie autoryzować endpointu.

SecurityConfig bez większych zmian

Reguły hasRole działają, jeśli authorities mają prefiks ROLE_

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()
    .requestMatchers("/health").permitAll()
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers(r ->
        r.getMethod().equals("POST") &&
        r.getRequestURI().startsWith("/api/vehicles")
    ).hasRole("ADMIN")
    .anyRequest().authenticated()
)
```

Jak to działa?

hasRole("ADMIN")
Spring porównuje z ROLE_ADMIN
Authority pochodzi z Role.name

Publiczne endpointy

/api/auth/login
/api/auth/register

DTO rejestracji

Do API nie wysyłamy encji User

```
public record RegisterRequest(  
    String login,  
    String password  
) { }
```

```
POST /api/auth/register  
Content-Type: application/json  
  
{  
  "login": "nowy",  
  "password": "test123"  
}
```

Data Transfer Object

Nie pokazujemy passwordHash
Nie pozwalamy klientowi ustawić roles
Domyślną rolę nadaje backend

UserService.register

Rejestracja znajduje ROLE_USER i zapisuje użytkownika

```
public void register(String login, String password) {
    if (login == null || login.isBlank()) {
        throw new IllegalArgumentException("Login nie może być pusty.");
    }
    if (password == null || password.isBlank()) {
        throw new IllegalArgumentException("Hasło nie może być puste.");
    }
    if (userRepo.findByLogin(login).isPresent()) {
        throw new IllegalArgumentException("Użytkownik już istnieje.");
    }

    Role userRole = roleRepository.findByName("ROLE_USER")
        .orElseThrow(() -> new IllegalStateException("Brak ROLE_USER."));

    User user = User.builder()
        .id(UUID.randomUUID().toString())
        .login(login)
        .passwordHash(passwordEncoder.encode(password))
        .roles(Set.of(userRole))
        .build();

    userRepo.save(user);
}

//pamiętamy o dodaniu encodera i repo ról w konstruktorze serwisu:
private final UserRepository userRepo;
private final RentalServiceInterface rentalService;
private final RoleRepository roleRepository;
private final PasswordEncoder passwordEncoder;

public UserService(UserRepository userRepo, RentalServiceInterface rentalService,
    RoleRepository roleRepository, PasswordEncoder passwordEncoder) {
    this.userRepo = userRepo;
    this.rentalService = rentalService;
    this.roleRepository = roleRepository;
    this.passwordEncoder = passwordEncoder;
}
```

Rejestracja

UserService, nie UserDetailsService

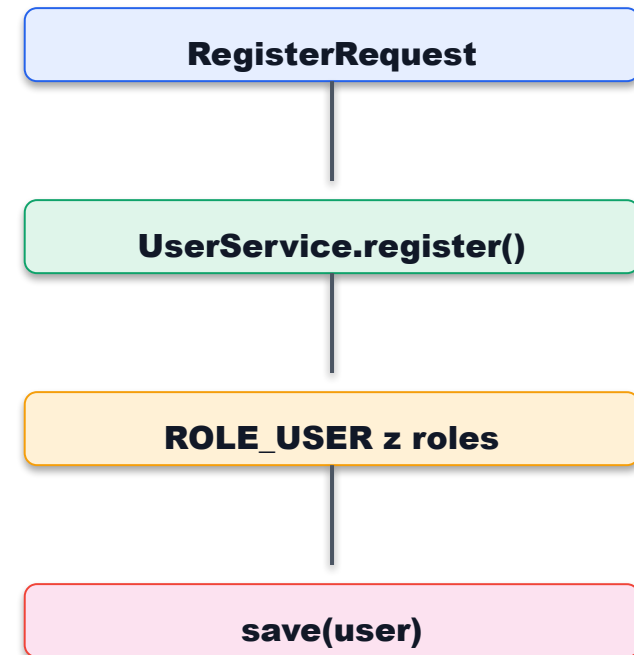
pamiętamy o dodaniu encodera i repo ról
w konstruktorze i klasie serwisu!

Klient nie wybiera roli
Hasło zawsze kodujemy
PasswordEncoderem
ROLE_USER musi istnieć w tabeli roles
JPA zapisze users i user_roles

AuthController: register

Endpoint rejestracji jest publiczny w /api/auth/**

```
@PostMapping("/register")
public ResponseEntity<?> register(
    @RequestBody RegisterRequest request) {
    try {
        userService.register(request.login(), request.password());
        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body("Registered successfully");
    } catch (IllegalArgumentException ex) {
        return ResponseEntity
            .badRequest()
            .body(ex.getMessage());
    }
}
```



Testy w Postmanie

Najpierw rejestracja, potem logowanie i request chroniony

1. Register

POST /api/auth/register
{ login, password }
Oczekiwane: 201 Created

2. Login

POST /api/auth/login
{ login, password }
Oczekiwane: token JWT

3. Request

Authorization: Bearer <token>
GET /api/vehicles
POST /api/vehicles tylko ADMIN

Nowy user:
roles: ["ROLE_USER"]

Admin:
roles: ["ROLE_ADMIN", "ROLE_USER"]

Checklist implementacji

Minimalna kolejność zmian dla wersji JPA

1. SQL: users bez role, nowe roles i user_roles
2. Role jako @Entity z name = ROLE_USER / ROLE_ADMIN
3. User.roles jako @ManyToMany przez user_roles
4. RoleRepository + RoleJpaRepository + adapter
5. MyUserDetailsService mapuje wszystkie role na authorities
6. JwtUtil zapisuje claim roles jako listę
7. UserService.register nadaje domyślnie ROLE_USER
8. Test: register → login → endpoint user/admin

Podsumowanie: po migracji Security nie opiera się na jednej kolumnie users.role, tylko na relacji users ↔ roles.